

LOOPGYM: LEARNING LOOP INVARIANTS WITHOUT SEEING THE VERIFICATION TARGET

Anonymous authors

Paper under double-blind review

ABSTRACT

Loop-invariant generators are commonly given the assertion or postcondition that their output must prove. This can simplify generation to copying or specializing to the target rather than recovering the behavior of the loop. We study *target-hidden loop verification*: the generator sees the precondition, guard, and loop body but not the verification target Q . It produces ACSL invariants I , and a result succeeds only when Frama-C/WP proves both that I is inductive and that $I \wedge \neg B \Rightarrow Q$. LOOPGYM trains the generator without querying Q . For supervised fine-tuning, multiple target-hidden responses are combined, optionally repaired, and filtered by Houdini; the verified invariant set becomes the target paired with the masked program. Reinforcement learning uses a separate execution signal: reachable states and three conservatively filtered kinds of synthetic negative examples. Rewards measure which negatives are rejected by the inductive clauses in each response, with an additional group-relative bonus for less common contributions. SFT data synthesis, RL, and inference use the same candidate normalization, union, optional repair, and Frama-C/WP-based Houdini filtering; only inference restores Q for the final proof. On 832 C programs, five-rollout inference verifies 74.76% of tasks, compared with 61.18% for the strongest external baseline. Reinforcement learning raises combine@10 from 52.40% to 56.97% and reduces the rollouts needed to reach 95% of the measured maximum from 30 to 10.

1 INTRODUCTION

Loop invariants summarize an unbounded computation in a finite assertion. For a loop `while (B) S` with precondition ϕ_{pre} and target Q , an invariant I must hold initially, be preserved by the loop body, and retain enough information at the exit to prove Q (Floyd, 1967; Hoare, 1969):

$$\phi_{\text{pre}} \Rightarrow I, \quad \{I \wedge B\} S \{I\}, \quad I \wedge \neg B \Rightarrow Q.$$

The first two conditions establish inductiveness, but they do not guarantee usefulness: *true* is often inductive and proves nothing about the loop exit.

Most invariant-synthesis systems are given Q during generation. Template search, counterexample-guided methods, and recent neural systems can therefore use the target or its failed proof obligations to guide the search (Gulwani et al., 2008; Gupta & Rybalchenko, 2009; Dillig et al., 2013; Alur et al., 2013; Garg et al., 2014; 2016; Si et al., 2020; Kamath et al., 2023; Cao et al., 2025). This is appropriate when the verification target is already known, but it also creates a shortcut: when Q is itself inductive, a generator can copy it into the loop without recovering the relations that explain the program.

We study *target-hidden loop verification*. The generator sees the precondition, loop guard, body, and in-scope variables, but all assertions and postconditions are removed. The original target Q is restored only after the invariant set has been fixed. Success still requires the complete Frama-C/WP proof above, so hiding the target changes the information available to generation without weakening the final criterion (§3).

This setting removes the most direct training signal. Inductiveness alone cannot distinguish a strong loop summary from a tautology or a loose bound, while using final proof success as a reward would expose the held-out target. LOOPGYM instead executes the loop to collect reachable

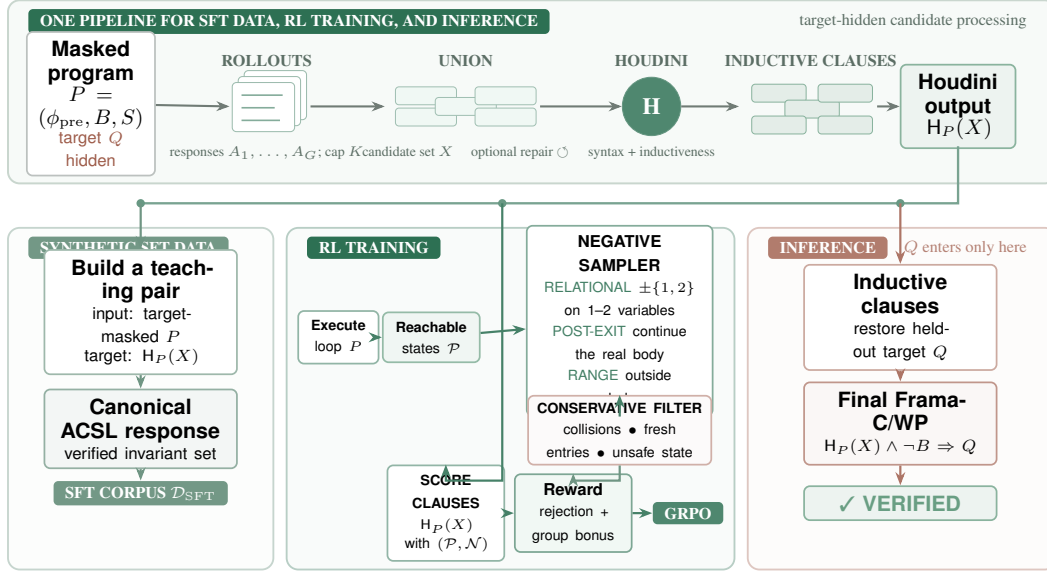


Figure 1: **Synthetic SFT data, RL training, and inference use the same candidate-processing pipeline.** Responses are capped, normalized, combined, optionally repaired, and filtered by Houdini on the target-masked program. The filtered invariant set becomes the target of an SFT pair. Separately, program executions provide positive and negative examples for the RL reward; these are not SFT data. Inference restores Q only for the final Frama-C/WP proof.

states and constructs three kinds of synthetic negative examples without consulting Q : relational perturbations of observed traces, continuations after genuine loop exits, and values outside sampled ranges. Conservative checks remove candidates that collide with observed executions, may be valid fresh entries, or depend on unsupported state.

Training has two distinct data paths. To construct synthetic SFT data, multiple target-hidden responses are normalized, merged, optionally repaired, and filtered by Houdini over Frama-C/WP. The masked program is paired with the resulting verified invariant set as a supervised example. Reinforcement learning uses the execution-derived positives and negatives described above to reward clauses that are both inductive and behaviorally informative. The same candidate-processing steps are used at inference, where Q is restored only for the final proof.

Our evaluation separates inference quality from the effect of reinforcement learning. On 832 integer C programs, five target-hidden responses with candidate union and Houdini verify 622 programs (74.76%), compared with 509 (61.18%) for the strongest external baseline. On the Qwen3-8B study, reinforcement learning increases combine@10 from 52.40% to 56.97% and reduces k_{95} , the number of responses needed to reach 95% of measured combine@100, from 30 to 10. A separate audit of 486,302 generated negative states finds no hard pair-level collision under an expanded execution check.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification and an evaluation protocol in which Q is unavailable to generation, feedback, and reward, but remains the final acceptance criterion.
- We use one verified candidate-processing pipeline to synthesize SFT pairs, compute execution-based RL rewards, and perform inference. The SFT targets are verified invariant sets; the separate RL signal uses conservatively filtered synthetic negatives.
- We evaluate inference, reinforcement learning, and negative-example quality separately, showing higher verification rates and substantially better rollout efficiency.

2 RELATED WORK

Programmatic invariant synthesis. Classical methods infer invariants with abstract interpretation, affine equalities, templates, interpolation, recurrences, or constraint solving (Karr, 1976; Cousot & Cousot, 1977; Cousot & Halbwachs, 1978; Sankaranarayanan et al., 2004; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017). These methods provide strong guarantees for selected languages, especially affine transition systems, but nonlinear templates trade generality for increasingly difficult algebraic search. Dynamic approaches instead infer likely relations from execution traces and then filter or prove them (Ernst et al., 2007; Nguyen et al., 2012; Garg et al., 2016). LOOPGYM also uses executions, but only to construct a training signal; final soundness is established independently by Frama-C/WP.

Neural and LLM-based invariant generation. Learned methods broaden the candidate language by using neural generators, counterexamples, or verifier feedback (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020). LOOPY generates ACSL clauses, filters them with Houdini, and repairs failures using Frama-C feedback (Kamath et al., 2023). CLAUSE2INV combines generated clauses under counterexample guidance, and IRANK learns to rank LLM-generated invariants before expensive verifier calls (Cao et al., 2025; Chakraborty et al., 2023). These systems solve a supplied verification problem. Our setting instead withholds its target throughout generation and training, then restores it only for final evaluation.

Specification generation. LLM-based tools also synthesize broader program specifications. AUTOSPEC repeatedly proposes and verifies ACSL annotations; SPECGEN combines LLM proposals with mutation in Java/JML; and SESPEC uses symbolic execution to generate goal-independent specifications for C (Wen et al., 2024; Ma et al., 2025; Yang et al., 2026). Such systems may generate preconditions, postconditions, predicates, and loop annotations. We study the narrower output of loop invariants and measure whether they are sufficient for a held-out target rather than whether they form a general behavioral specification.

Reinforcement learning from formal feedback. Verifier-guided reinforcement learning has been used for program synthesis and formal reasoning. RE:FORM uses scalable curation and verifier-guided RL for Dafny, while SPECRL generates impossible input–output pairs and rewards specifications that reject them (Yan et al., 2025; Huang et al., 2026). The latter shares our motivation that validity alone does not distinguish a useful specification from a weak one. LOOPGYM specializes this idea to loops: the target is unavailable as a reward label, synthetic negative states provide a strength signal, and Frama-C/WP separately proves inductiveness.

3 TASK DEFINITION

3.1 VERIFICATION OBJECTIVE

A verification instance is a pair (P, Q) . The program fragment $P = (\phi_{\text{pre}}, B, S)$ contains the precondition, loop guard, and loop body; Q is an assertion or postcondition that must hold after the loop. Let $\mathcal{R}(P)$ denote the states reachable at the loop head from inputs satisfying ϕ_{pre} .

The generator returns a conjunction I of ACSL loop-invariant clauses. Frama-C/WP must prove initiation and preservation,

$$\phi_{\text{pre}} \Rightarrow I \quad \text{and} \quad \{I \wedge B\} S \{I\}.$$

It must also prove the original verification goal,

$$I \wedge \neg B \Rightarrow Q.$$

The third condition matters because inductiveness alone admits predicates that are correct but too weak to establish Q .

3.2 INFORMATION AVAILABLE TO THE GENERATOR

Definition 1 (Target-hidden loop verification). *The framework receives (P, Q) , but the generator and all intermediate feedback receive only P . The generated invariant set is accepted only if it satisfies all three obligations above after the original Q is restored.*

Before generation, complete ACSL `assert` and `ensures` clauses, executable assertion calls, and conventional error-label names are removed. Preconditions and executable loop semantics remain visible. Candidate generation, repair feedback, execution sampling, and Houdini filtering all use this target-masked program. The untouched full program is used only for final verification.

The restriction concerns information flow, not output syntax: both target-visible and target-hidden systems produce the same ACSL `loop invariant` clauses. Querying final proof success during training would reveal information about Q , so LOOPGYM instead derives its reward from program executions (§4.4).

3.3 SCOPE AND MOTIVATING EXAMPLE

We focus on a single braced `while` loop over scalar C integers. Arithmetic equalities and inequalities in this domain can be checked automatically, allowing us to study invariant discovery without the additional representation choices required by arrays, heaps, or recursive structures. Nonlinear updates remain challenging because exact algorithmic guarantees are largely concentrated on affine transition systems.

Consider the following abridged program:

```
1 int x = 1, y = 1, c = 1;
2 while (c < k) {
3   c = c + 1;
4   x = x*z + 1;
5   y = y*z;
6 }
7 /*@ assert 1+x*z-x-z*y == 0; */
```

When the assertion is visible, the clause `loop invariant 1+x*z-x-z*y == 0;` can be copied directly. When it is hidden, the same nonlinear relation must be recovered from the coupled updates to `x` and `y`. This example illustrates the difference between proving a supplied candidate and discovering the loop relation needed by a held-out target.

4 METHOD

4.1 OVERVIEW

For a verification instance (P, Q) , the generator receives the target-masked program $P = (\phi_{\text{pre}}, B, S)$. One sampled response, or *rollout*, $A = (a_1, \dots, a_L)$, is an ordered list of parsed invariant clauses. A training group $\mathcal{A} = \{A_1, \dots, A_G\}$ contains G responses for the same program. Only the first K clauses of each response are used:

$$\bar{A} = (a_1, \dots, a_{\min(L, K)}), \quad o(A) = \max(0, L - K).$$

Given a set of clauses X , $H_P(X)$ denotes the subset retained after syntax checking and Houdini filtering with Frama-C/WP. Execution provides reachable loop-head states $\mathcal{P}$ and synthetic negative traces $\mathcal{N}$. A negative trace may contain several witness states from one perturbed execution. We write $\text{rej}(X) \subseteq \mathcal{N}$ for the negative traces on which at least one clause in $H_P(X)$ is false.

The method has four parts. First, candidate responses are normalized, combined, optionally repaired, and filtered by the verifier. This pipeline is used to construct synthetic SFT pairs. Program executions then provide a separate target-independent signal for reinforcement learning. Inference reuses the candidate pipeline without SFT serialization, execution sampling, or reward computation.

4.2 CANDIDATE GENERATION AND VERIFICATION

The model emits scalar ACSL clauses of the form `loop invariant <expr>;`. Each response is capped at K parsed clauses, normalized, and deduplicated. For a group of responses, the initial

candidate set is

$$X_0 = \bigcup_{i=1}^G \bar{A}_i.$$

An optional repair round receives only syntax, scope, establishment, and preservation verdicts from the target-masked program. Its response R_t can add clauses but cannot remove existing candidates:

$$X_{t+1} = X_t \cup R_t.$$

The filter $H_P(X)$ first removes malformed invariant lines and then runs Houdini over Frama-C/WP (Flanagan & Leino, 2001; Kirchner et al., 2015). Houdini repeatedly removes clauses whose establishment or preservation obligations fail and returns an inductive subset. During reward computation, sampled reachable states provide a cheap pre-filter for out-of-scope or obviously false clauses. Inference skips this optimization. In both cases, Frama-C/WP is the source of soundness.

Training evaluates $H_P(\bar{A}_i)$ for each response, $H_P(X_0)$ for the union, and $H_P(X_t \cup R_t)$ when learning repair. Inference applies the same cap, normalization, union, repair feedback, and Houdini filter. The difference is downstream: training computes rewards from execution examples, whereas inference inserts the filtered clauses into the full program and checks the restored Q .

4.3 SYNTHETIC DATA FOR SUPERVISED FINE-TUNING

For each target-masked program P , the data generator samples a group of candidate responses and applies the same cap, normalization, union, optional repair, and Houdini filter defined above. When the resulting set $H_P(X)$ is nonempty, it is serialized as a canonical ACSL response and paired with P :

$$\mathcal{D}_{\text{SFT}} = \{(P, \text{serialize}(H_P(X)))\}.$$

The original target Q is not included in the input, feedback, or output of this construction. Supervised fine-tuning therefore teaches the model to imitate verifier-accepted invariant sets produced by the same aggregation and filtering procedure later used at inference.

This synthetic SFT corpus is distinct from the execution examples introduced next. An SFT target is a verified ACSL invariant response; a positive or negative execution state is used only to compute the reinforcement-learning reward and is never serialized as an SFT answer.

4.4 EXECUTION-BASED TRAINING EXAMPLES

The execution sampler never reads an assertion or postcondition. It supports one braced `while` loop over scalar integer parameters, locals, and file-scope variables. It instruments the loop, compiles it with `gcc`, and records loop-head valuations for a deterministic, precondition-checked input sweep. The recorded states form the positive set $\mathcal{P}$. Runs continue to a genuine loop exit; hitting the iteration cap marks a run as incomplete and disables any construction that requires reliable terminal or range information.

The sampler constructs three kinds of negative examples:

- **Relational perturbations.** Single-variable and pairwise $\pm\{1, 2\}$ changes are applied to densely observed states. These examples target relations such as $x + y = n$ that may hold even when each variable has a broad range.
- **Post-exit continuations.** After an observed genuine exit, the loop body is executed for additional steps. The continuation preserves the program’s transition relation while moving beyond the actual run.
- **Range perturbations.** One variable is moved beyond the range observed for that variable. These examples test whether a candidate captures useful bounds.

These are training candidates, not proofs of global unreachability. Any candidate equal to an observed reachable state is removed. The sampler also removes states that may be feasible fresh loop entries for the recorded input, and it excludes variables affected by unsupported or oracle-dependent behavior. Unsupported block-local state, pointers, arrays, or function calls disable synthetic negatives; if no safe variable remains, the program contributes positive states only. Exact sampling schedules, budgets, perturbation sizes, and fallback conditions are reported in Appendix A.

4.5 REWARD AND TRAINING

For response A_i , let $R_i = \text{rej}(\bar{A}_i)$ be the negative traces rejected after Houdini filtering. Its negative rejection rate is

$$\text{coverage}(A_i) = \frac{|R_i|}{|\mathcal{N}|}.$$

To reward complementary responses, define the frequency with which a negative trace n is rejected in the group and a bonus for less common rejections:

$$f(n) = \sum_{j=1}^G \mathbf{1}[n \in R_j], \quad (1)$$

$$\text{bonus}(A_i) = \frac{1}{|\mathcal{N}|} \sum_{n \in R_i} \left(1 - \frac{f(n)}{G}\right). \quad (2)$$

The first term rewards overall strength; the second rewards a response for contributing behavior not already covered by most of its peers. A response need not contain a complete proof to receive credit.

We do not penalize a unique clause merely because it adds no independently observed coverage: it may support another clause or be needed for the hidden target. We only count conservative syntactic and local algebraic duplicates,

$$d_{\text{dup}}(A) = |\bar{A}| - |\text{dedup}(\bar{A})|.$$

The deduplication key normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association. It deliberately avoids transformations whose validity depends on stronger arithmetic assumptions.

The generation reward is

$$r_{\text{gen}}(A) = w_c \text{coverage}(A) + w_g \text{bonus}(A) - w_d d_{\text{dup}}(A) - w_o o(A), \quad (3)$$

with default weights $(w_c, w_g, w_d, w_o) = (1.0, 0.3, 0.02, 0.05)$. Clauses after the cap K receive no coverage and incur the overflow cost. Malformed and non-inductive clauses are removed before reward computation, so one failed clause does not erase useful clauses in the same response.

The union also receives the diagnostic rejection rate $|\text{rej}(X_0)|/|\mathcal{N}|$, which is used to decide whether another group should be sampled. If no negative examples can be constructed safely, the reward falls back to binary inductiveness: a nonempty response receives one only when all of its retained clauses pass Houdini.

Repair is trained separately. For a current candidate set X and capped repair response R_i , its reward is the improvement in negative rejection:

$$\text{gain}_i = \text{coverage}(X \cup R_i) - \text{coverage}(X), \quad (4)$$

$$r_{\text{ref},i} = \text{gain}_i - w_o o(R_i). \quad (5)$$

The existing set X is not truncated. Generation and repair examples are mixed in the same policy update, training the two model actions used during deployment.

4.6 INFERENCE

At inference, the model receives the target-masked program and returns n candidate responses. Each is capped at K clauses, and the normalized clauses are combined into X_0 . Inference does not construct positive or negative execution examples.

When repair is enabled, Frama-C/WP reports target-independent syntax, scope, establishment, and preservation failures. A stateless prompt asks the model to repair failed clauses or add supporting invariants. Each repair is capped before it is added to the candidate set. Repair stops when all clauses pass, the set reaches a fixed point, or the round budget is exhausted.

Finally, H_P filters the accumulated candidates on the target-masked program. The retained invariants are inserted into the untouched full program, Q is restored, and Frama-C/WP checks every invariant and target obligation. If verification fails, the system may sample another response group up to its configured retry budget. No sampled state or training reward participates in this final decision.

5 EXPERIMENTS

We evaluate three questions in the order a reader encounters the system: (1) how effective and costly is target-hidden inference, (2) whether reinforcement learning improves that inference procedure, and (3) why candidate aggregation and execution-based rewards are useful.

5.1 EXPERIMENTAL SETUP

Benchmarks. The evaluation contains 832 integer C programs. The *Core-366* subset contains 316 linear programs from Code2Inv, SyGuS, and SV-COMP and 50 nonlinear programs from LIPuS. The remaining 466 programs are the integer fragment of Loopy’s original 469-program corpus; three floating-point programs are excluded because they fall outside our scalar-integer predicate language. Every method uses the same fixed manifest, and unsupported inputs remain failures in the common denominator.

Target-hidden evaluation. Before any model-facing step, we remove ACSL assert and ensures predicates, executable assertion calls, and conventional error-label names. Preconditions and executable loop semantics remain visible. After a method fixes its invariants, they are inserted into the untouched original program. A task succeeds only if the source parses, Frama-C/WP proves establishment and preservation for every retained invariant, and all restored target goals succeed. Failures, timeouts, malformed outputs, and unsupported programs count as zero.

Verifier, models, and baselines. All final checks use Frama-C 32.1 (Germanium), WP, and Z3 4.15.4. The main inference comparison fixes `gpt-5-nano` across all LOOPGYM variants, so differences among those rows come only from inference. We compare with the native target-hidden pipelines of AUTOSPEC, SESPEC, and CLAUSE2INV, and with Daikon 5.8.24 (Wen et al., 2024; Yang et al., 2026; Cao et al., 2025; Ernst et al., 2007). CLAUSE2INV supports only the 366 programs with transition verification conditions; its remaining inputs still count as failures.

The model-scale analysis samples 100 target-hidden responses per task from the official Qwen3-4B, Qwen3-8B, Qwen3-14B, and Qwen3-30B-A3B checkpoints (Yang et al., 2025), at temperature 1.0 with thinking disabled. The reinforcement-learning comparison uses Qwen3-8B and the corresponding 8B-RL-Zero checkpoint under the same 100-response protocol.

Metrics. For a program with c independently successful responses among $n = 100$, direct `pass@k` uses the unbiased estimator $1 - \binom{n-c}{k} / \binom{n}{k}$. `Combine@k` instead unions the clauses from the first k responses, applies Houdini, and tests the resulting invariant set against the restored target. We summarize response efficiency with

$$k_{95} = \min\{k : C_{\pi}(k) \geq 0.95 C_{\pi}(100)\},$$

where $C_{\pi}(k)$ is `combine@k` for policy π .

5.2 RQ1: HOW EFFECTIVE IS TARGET-HIDDEN INFERENCE?

NAIVE uses one response and a minimal prompt. R1-NOH uses the LOOPGYM prompt but no Houdini filtering; R1-H adds Houdini; and R5-H/R10-H combine five or ten target-hidden responses before the same Houdini and final proof. Retries and repair are disabled in this comparison.

Method	Linear / 316	NLA / 50	Loopy / 466	All / 832	Time (s)	Exact tokens	Est. tokens
AUTOSPEC	203	4	302	509 (61.18%)	64.63	—	2,031.54
CLAUSE2INV	63	1	0	64 (7.69%)	29.04	—	385.80
DAIKON	73	0	101	174 (20.91%)	4.89	0	—
SESPEC	180	3	231	414 (49.76%)	117.31	44,807.05	—
NAIVE	154	4	221	379 (45.55%)	28.87	4,493.37	—
R1-NoH	193	7	252	452 (54.33%)	53.52	5,849.64	630.14
R1-H	203	7	294	504 (60.58%)	66.60	6,928.30	—
LOOPGYM (R5-H)	250	7	365	622 (74.76%)	284.81	33,404.56	—
R10-H	255	7	376	638 (76.68%)	536.81	68,953.35	—

Table 1: Target-hidden verification results and mean generation cost per task. Exact and estimated token averages are reported separately. R1-NoH’s time and exact-token means cover Loopy-466, while its estimate covers Core-366. Unsupported Clause2Inv inputs remain failures in the 832-program denominator.

Table 1 shows that five-response inference verifies 622 programs, 113 more than AUTOSPEC, the strongest external baseline. The controlled rows isolate the contributions of prompting, Houdini, and response aggregation: 379/832 for NAIVE, 452/832 for R1-NoH, 504/832 for R1-H, and 622/832 for R5-H.

The improvement has a clear cost. R5-H averages 33,404.56 exact tokens and 284.81 seconds per task. Increasing the budget to ten responses adds only 16 proofs while raising the averages to 68,953.35 tokens and 536.81 seconds. This diminishing return motivates improving the generator rather than scaling the response budget indefinitely.

5.3 RQ2: DOES REINFORCEMENT LEARNING IMPROVE INFERENCE?

We compare Qwen3-8B with 8B-RL-Zero while holding the candidate union, Houdini filter, target-hiding protocol, and response budget fixed.

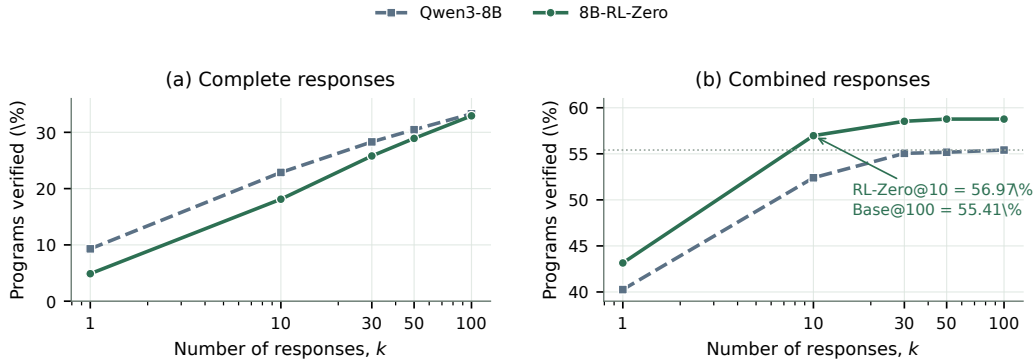


Figure 2: Reinforcement learning improves combined inference even though it does not improve the chance that one response proves the target. With ten responses, 8B-RL-Zero exceeds the base model’s combine@100 result.

Figure 2 shows different trends for complete responses and complementary clauses. RL-Zero lowers direct pass@1 from 9.30% to 4.88% and leaves direct pass@100 nearly unchanged (33.29% versus 32.93%). Under the deployed combination procedure, however, combine@10 rises from 52.40% to 56.97%, and combine@100 rises from 55.41% to 58.77%. Ten RL-Zero responses outperform 100 base-model responses, while k_{95} falls from 30 to 10.

The trained policy therefore supplies useful complementary clauses with fewer generations. This is the behavior encouraged by the group-relative reward: each response can contribute clauses that

survive Houdini and cover behavior missed by its peers, even if the response is not a complete proof by itself.

5.4 RQ3: WHY ARE AGGREGATION AND EXECUTION-BASED REWARDS NEEDED?

Candidate aggregation. Inductiveness by itself cannot distinguish a useful invariant from `true`, and a binary whole-response reward discards useful clauses when another clause in the same response fails. Reinforcement learning with verifiable rewards can also concentrate probability on already rewarded paths while reducing large- k exploration (Yue et al., 2025). This matters here because different responses often contain complementary parts of a proof.

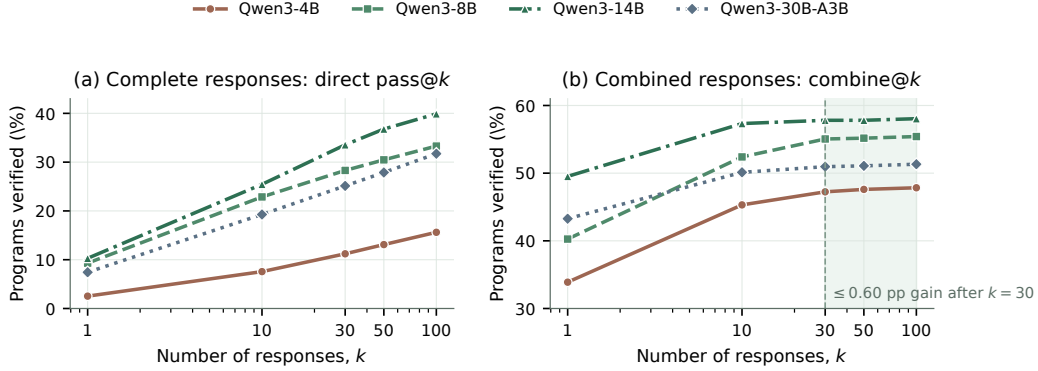


Figure 3: Direct pass@ k measures complete responses, whereas combine@ k measures whether the accumulated clauses contain a sufficient Houdini-filtered subset. For all four official Qwen3 checkpoints, the combine curves change by at most 0.60 percentage points after $k = 30$.

Figure 3 shows that partial responses contain substantial value. Direct pass@100 is 15.62% for Qwen3-4B, 33.29% for 8B, and 39.90% for 14B. Combining clauses raises 4B to 47.84% and 14B to 58.05%. Houdini can recover compatible clauses from imperfect responses, but it cannot invent a relation that the model never emits.

The curves also saturate early: no checkpoint gains more than 0.60 percentage points between combine@30 and combine@100. A useful reward should therefore increase both the attainable verification rate and the rate at which new responses contribute previously missing clauses.

Reward and negative-example checks. Across ten hand-written specifications, every gold invariant survives the verifier filter and rejects at least 99.6% of the intended negative examples, with no ordering violations in the reward sanity checks. A larger audit on Core-366 compares 486,302 generated negative states against an expanded set of program executions and finds no hard pair-level collision. These checks do not prove that every synthetic negative is globally unreachable, but they support the conservative construction used for training.

6 CONCLUSION AND LIMITATIONS

We studied loop-invariant generation when the assertion or postcondition is hidden from the generator. LOOPGYM uses the same verified candidate pipeline to synthesize SFT pairs, train with execution-based rewards, and perform inference. SFT targets are Houdini-filtered invariant sets paired with masked programs. Separately, reachable executions and conservatively filtered negative states provide the reinforcement-learning signal. The original target is restored only for final verification.

The experiments show that combining multiple target-hidden responses is substantially more effective than relying on a single complete response, and that reinforcement learning improves both the attainable combined verification rate and response efficiency. These results support evaluating learned invariant generators under the candidate-combination procedure actually used at deployment, rather than by standalone pass rate alone.

The current implementation is limited to one loop over scalar integer state. Its synthetic negatives are conservatively checked training examples, not proofs of global unreachability, and final soundness still depends on Frama-C/WP. Extending the approach to multiple loops, arrays, heaps, richer assertion languages, and inter-loop properties remains future work (Liu et al., 2024).

REFERENCES

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.
- Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.
- Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proc. 5th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 84–96, 1978.
- Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 443–456, 2013.
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.
- Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.
- Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.
- Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 499–512, 2016.
- Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 281–292, 2008.
- Ashutosh Gupta and Andrey Rybalchenko. InvGen: An efficient invariant generator. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 634–640. Springer, 2009.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.

-
- Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.
- Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.
- Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.
- Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- Chang Liu, Xiwei Wu, Yuan Feng, Qinxiong Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.
- ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pp. 683–693, 2012.
- Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.
- Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 318–329, 2004.
- Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.
- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.
- Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

594 Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin
595 Gu. Integrating symbolic execution with LLMs for automated generation of program specifications.
596 *arXiv preprint arXiv:2506.09550*, 2026.
597
598 Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop
599 invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming*
600 *Language Design and Implementation (PLDI)*, pp. 106–120, 2020.
601
602 Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang.
603 Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?
604 In *Advances in Neural Information Processing Systems*, volume 38, 2025.
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647

A IMPLEMENTATION DETAILS

Table 2 lists the implemented defaults. Execution sampling is used only to compute reinforcement-learning rewards; it does not construct the synthetic SFT corpus. SFT synthesis, RL, and inference use the same response cap, candidate union, repair format, and Houdini implementation. Inference has its own repair budget and does not construct execution examples.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
range-perturbation steps	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	320 (≤ 64 fallback) / 64 / 128	relational / post-exit / range examples; stable round-robin within each kind
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	(1.0, 0.3)	negative rejection / group-relative bonus weights
w_d	0.02	cost per conservative duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generation/refinement response
w_o	0.05	overflow cost per clause after K
re-roll threshold	0.6	group batch score below which re-rolling is advised
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget
m_{refine}	0	inference refinement disabled by default; any non-negative round budget is supported

Table 2: Defaults for execution sampling (§4.4), reward computation (§4.5), inference, and verification.

The exact near-input tier order is 0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9.

B GENERATION PROMPT

The same generation template is used to propose candidates for SFT synthesis, RL, and inference. Any assertion is stripped before the program is inserted. General ACSL rules are supplied separately by `system_prompt.txt`.

Generation prompt

You are given a C function. Produce the strongest inductive ACSL loop invariants that capture the loop’s behavior (conservation/relational laws + tight progress bounds). Output ONLY invariant lines, each formatted

```

exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}

```

C TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, Frama-C/WP, Why3, and Z3. It exposes POST `/reward`, accepting program source and an array of model responses. The implementation retains the following field names for compatibility:

```

{program, n_positives, n_negatives, filter_mode,
 reward_mode, rollout_rewards[], base[], hard_bonus[],
 redundant_clauses[], redundancy_penalty[],
 generated[], accepted[], overflow[], overflow_penalty[],
 rollouts[], batch_score, should_reroll}

```

Responses may be raw LLM text, invariant lists, or annotated code; parsing is uniform. For refinement training, POST `/refine_feedback` returns the WP verdict table and assembled stateless prompt, while POST `/refine_reward` returns both the nonnegative improvement diagnostic (stored as `delta_base`) and `refine_rewards[]` after overflow cost. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.